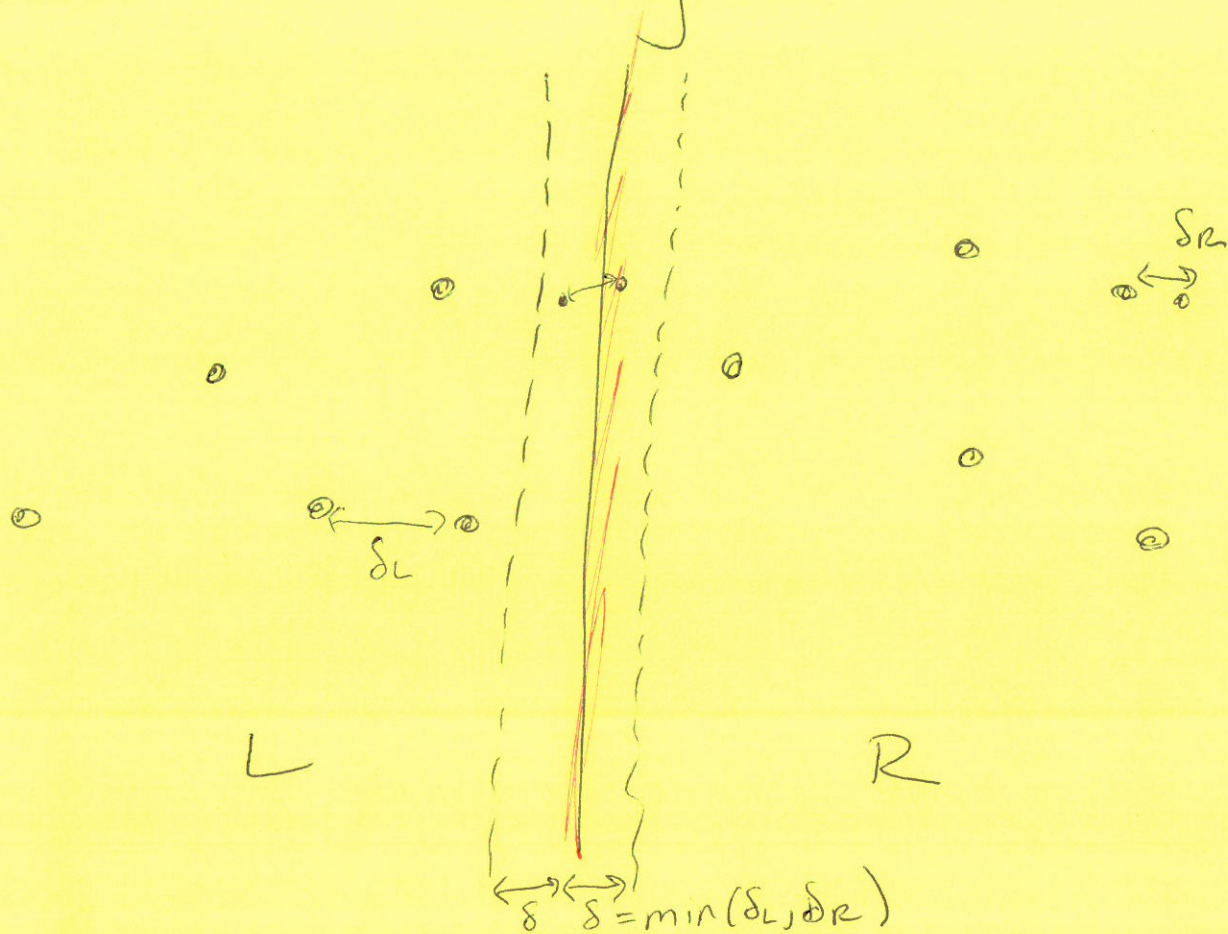


10 March 2026

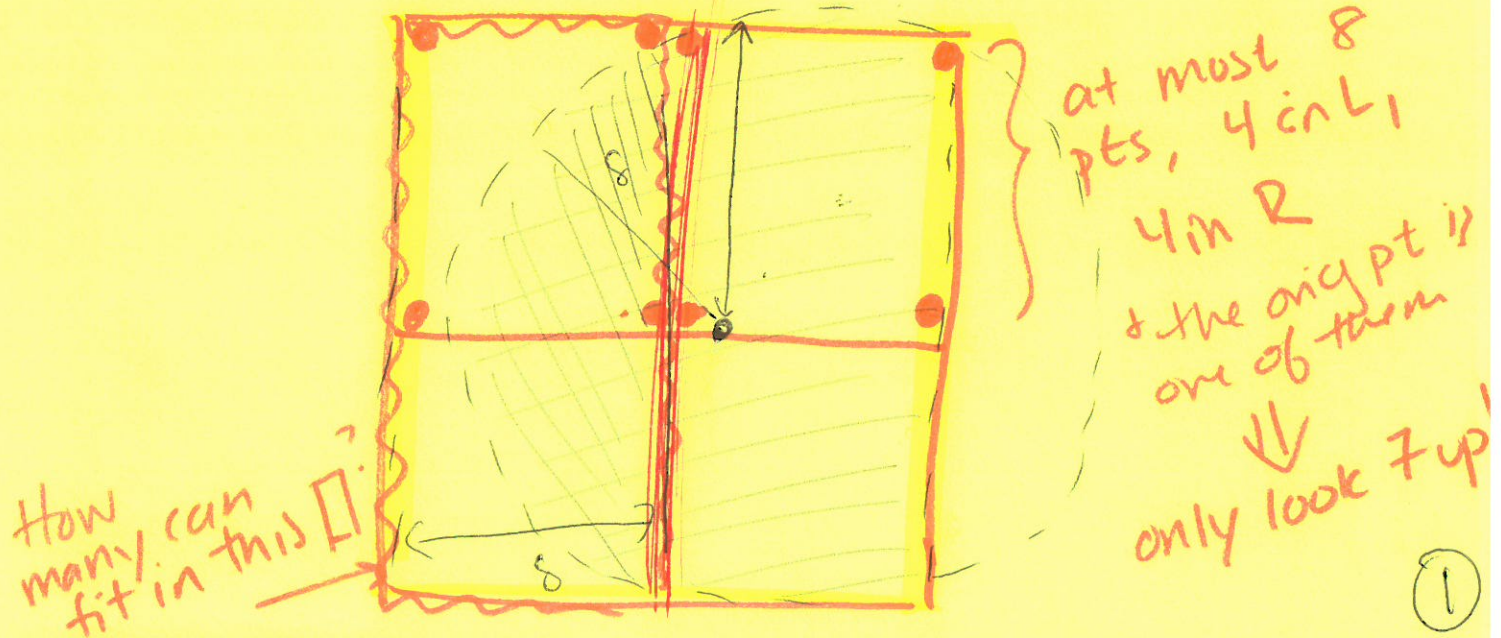
Closest Pair - the key move



case 1: $\exists$ a closest pair s.t. both are in L

case 2: " " R

case 3: neither $\Rightarrow$ one is L + one in R



CSCI 432 Handout 08: Dynamic Programming

Name: CLASS

Collaborators: _____

17 February 2026

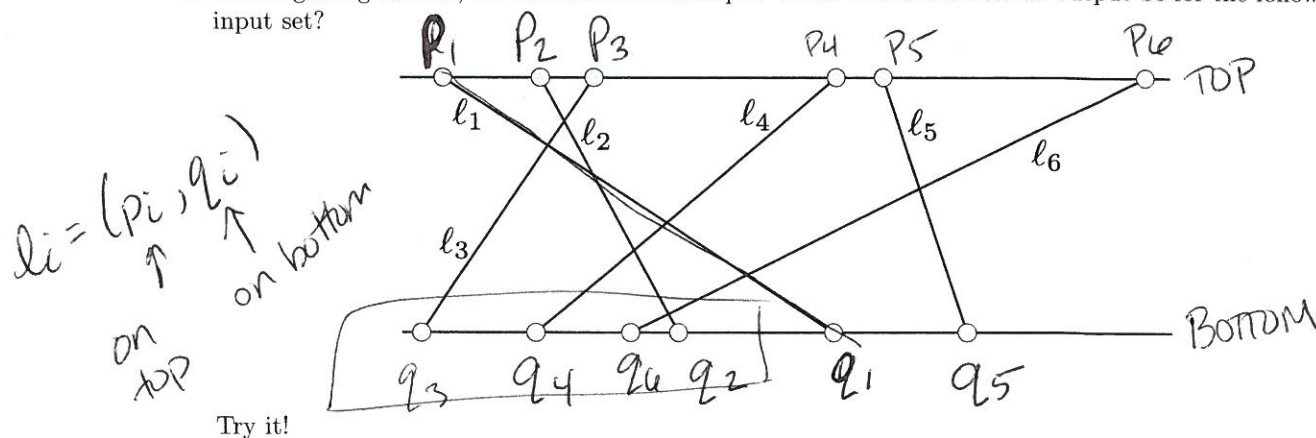
The Problem

Suppose we are given a set L of n line segments in the plane, where each segment has one endpoint on the line $y = 0$ and one endpoint on the line $y = 1$, and all $2n$ endpoints are distinct. Describe and analyze an algorithm to compute the largest subset of L in which no pair of segments intersects.

From JE, Chapter 3, Problem 19 (a).

Practice

1. Before getting started, let's consider an example. What would the desired output be for the following input set?



The recursion

consider inclusion based on order from top

NOT including (p_1, q_1)

→ next subproblem: $L \setminus \{(p_1, q_1)\}$

$$= L[2 : 6]$$

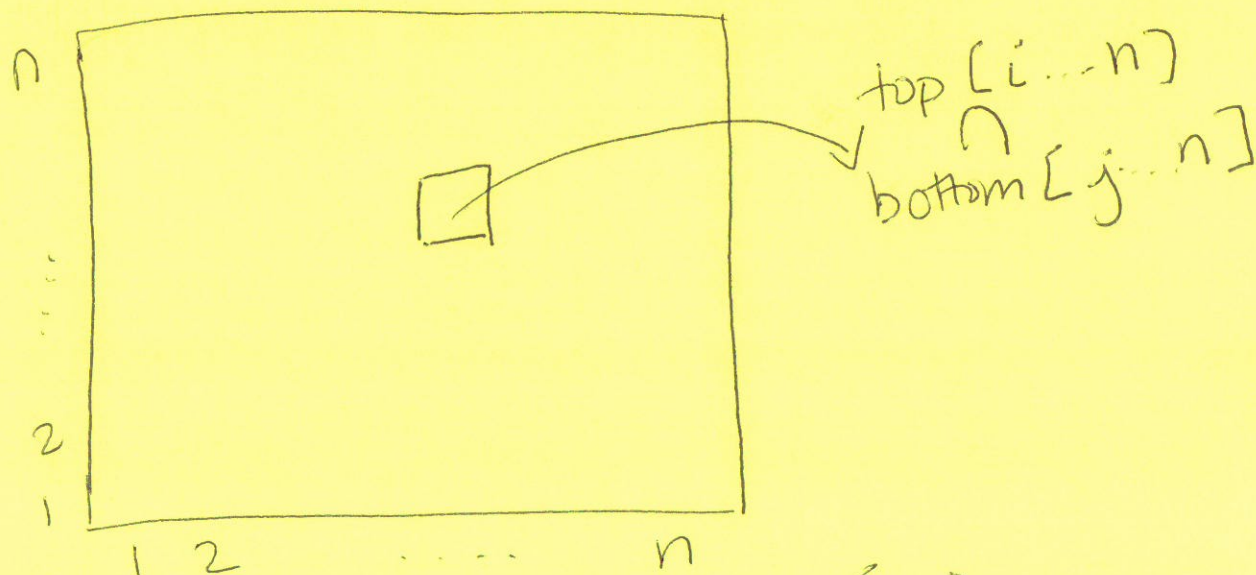
↑ b/c indexed based on top

Do ~~not~~ include (p_1, q_1)

→ next subproblem: $L \setminus \{(p_1, q_1), l_3, l_4, l_6, l_2\}$

$$= \{q_5\}$$

when sorted by bottom: removes all to the left of q_1 .



i , indexing into $\{p_i\}$, the top

Alt: on bottom, look for longest increasing subsequence.

(3)

Depth-first search is just one (perhaps the most common) species of a general family of graph traversal algorithms that I call *whatever-first search*. The generic traversal algorithm stores a set of candidate edges in some data structure that I'll call a "bag". The only important properties of a "bag" are that we can put stuff into it and then later take stuff back out. A stack is a particular type of bag, but certainly not the only one. Here is the generic algorithm:

```

WHATEVERFIRSTSEARCH(s):
  put s into the bag
  while the bag is not empty
    take v from the bag
    if v is unmarked
      mark v
      for each edge vw
        put w into the bag

```

← an umbrella algorithm for DFS, BFS & more!

I claim that WHATEVERFIRSTSEARCH marks every node reachable from s and nothing else. The algorithm clearly marks each vertex in G at most once. To show that it visits every node in a connected graph at least once, we modify the algorithm slightly; the modifications are in bold red. Instead of keeping vertices in the bag, the modified algorithm stores pairs of vertices. This modification allows us to remember, whenever we visit a vertex v for the first time, which previously-visited neighbor vertex put v into the bag. We call this earlier vertex the *parent* of v .

```

WHATEVERFIRSTSEARCH(s):
  put ( $\emptyset$ , s) in bag
  while the bag is not empty
    take (p, v) from the bag      (*)
    if v is unmarked
      mark v
      parent(v) ← p
      for each edge vw          (†)
        put (v, w) into the bag  (**)

```

adds this explicitly but not really needed for the algo to work

Lemma 5.1. *WHATEVERFIRSTSEARCH(s) marks every vertex reachable from s and only those vertices. Moreover, the set of all pairs $(v, \text{parent}(v))$ with $\text{parent}(v) \neq \emptyset$ defines a spanning tree of the component containing s .*

Proof: First we argue that the algorithm marks every vertex v that is reachable from s , by induction on the shortest-path distance from s to v . The algorithm marks s . Let v be any other vertex reachable from s , and let $s \rightarrow \dots \rightarrow u \rightarrow v$ be any path from s to v with the minimum number of edges. (There must be such a path, because v is reachable from s .) The prefix path $s \rightarrow \dots \rightarrow u$ is shorter than the shortest path from s to u , so the inductive hypothesis implies that the

4 Why? The Loop Invariant

→ Proves Partial correctness
"If the algo terminates,
then it is correct"

Finally, we prove why the algorithm works. Typically, this is done with a loop (or recursion) invariant. Here, we focus on setting up the loop invariant. What are the following statements (start with your best guess, then come back and revise if needed):

The loop guard G : what keeps you in the loop

$G = \text{"the bag is not empty"}$ $\neg G = \text{"the bag is empty"}$

The post-condition Q : What the loop was supposed to do.

$Q = \text{visit every vertex reachable from } s$

The pre-condition P : what is true before entering the loop

The loop invariant $L = L_i$: find L such that the I, M, E conditions are satisfied

Hint: incremental correctness. what is going well so far?

Next, we use this loop invariant (and other statements above) to prove partial correctness. There are three parts to partial correctness: Initialization, Maintenance, and End.

Initialization: If the preconditions are met and if we enter the loop, then the loop invariant is correct. $(P \wedge G \Rightarrow L)$.

Answer

Proof (often vacuisty true, or a simple proof)

Maintenance: If we entered the loop and the loop invariant held, then when we exit the loop, the loop invariant will still hold: $(G \wedge L_i \Rightarrow L_{i+1})$.

Answer

Proof (walk through algorithm)

End: If the loop ends and the loop invariant was correct, then the algorithm is correct. $(L \wedge \neg G \Rightarrow Q)$.

Answer

Proof (hopefully falls out)

A simpler loop invariant:

A	8	π	10	
	1	2	3	

$$\text{max} = \cancel{A[1]} A[3] \dots$$

Loop invariant: $\text{max} = \text{max value in } A[1 \dots i]$.

```
i = 0 ; max = -∞
while i < n
|   if A[i] > max
|   |   max ← A[i]
|   end if
end while
```